

Exercise Sheet 7 - Principal Component Analysis

Deadline for submission: Wednesday, 3 December 2024 at 8:00.

Discussed in exercise session: Friday, 5 December 2024 at 10:15.

This exercise sheet will be about PCA and two example applications of the method. Make sure to familiarize yourself with the concept and review the Singular Value Decomposition if necessary. For getting an more intuitive idea of the PCA decomposition, you can take a look at the interactive examples given here: <https://setosa.io/ev/principal-component-analysis/>. A more formal tutorial on PCA is given in the reference [1].

Exercise 7.1 - Visualizing High-Dimensional Data

As we can use PCA to encode/project our data x_i onto a **lower-dimensional** vector z_i , it offers itself for visualizing high-dimensional data by plotting the most important directions of the encoding instead of the raw data. PCA-based visualization has become a standard practice for exploratory analysis of high-dimensional data. It can be used to learn about patterns in the data, such as clusters or whether the data separates well for predicting a given label y .

In this exercise, you are given data from a Breast Cancer Diagnosis task. The data set contains 32 different attributes extracted from image data and a label containing the diagnosis (malignant or benign). The goal is to visualize the multi-dimensional data in just one plot and see whether there is any pattern with respect to the label.

Complete the following tasks in the given Jupyter notebook `PCA_visualization_empty`.

1. As a warm-up exercise, answer the preliminary work questions.
2. Implement PCA from scratch (using SVD) and visualize the projected data. Can you see any patterns with respect to y ? Do you think the given data contains information about the target?
3. Compare your results with the PCA results you get from using the scikit learn method `sklearn.decomposition.PCA`.
4. Sometimes the encoded z flips its sign for different implementations of PCA (i.e. the plot of the low-dimensional data is mirrored along an axis). What could be the cause of this?

Exercise 7.2 - Latent Semantic Analysis

Latent Semantic Analysis (LSA) is a technique in natural language processing for extracting topics from a set of documents. It uses a document-term matrix, where each entry represents the frequency of a word in a document. By applying PCA to this matrix, we can capture the main “topics” in the corpus in the form of principal components.

We will look at a small document dataset and see how does PCA result in a better similarity measure between two documents.

Follow the instructions and complete the code in the given Jupyter Notebook. You can use the scikit-learn PCA method (`sklearn.decomposition.PCA`).

1. Compute the document-term matrix X , where each entry x_{ij} represents the frequency of word j in document i .
2. Use PCA to reduce the dimensionality of X by keeping only the top k principal components.
3. Project each document onto the reduced-dimension space, creating a new vector representation for each document based on the principal components. To compare the similarity measures resulted from the original space and PCA space, we will conduct clustering and visualize the results. Compare the two clustering results, and what can you say about the results?

Exercise 7.3 - Eigenfaces

A well known method for classification of images is based on so-called “Eigenfaces” [2]. In this exercise we will use PCA to obtain the Principal Components of high-dimensional Image data, which we can again visualize as images (those are the “Eigenfaces”). Note that in this method, we visualize the Eigenvectors - not the projections.

We will look at grayscale images with a width of 62 pixels and a height of 47 pixels. Each image can be flattened into a $62 * 47 = 2914$ dimensional feature vector x_i , with values ranging from 0 to 255. We can then arrange these column vectors into a matrix X , and apply the PCA on it.

Follow the instructions in the given Jupyter Notebook `Eigenfaces.empty.ipynb` and complete the code. You can use the scikit-learn PCA method.

1. Compute the PCA of the data and visualize the first 30 Eigenfaces (i.e. the principal components corresponding to the largest 30 eigenvalues in descending order).
2. Reconstruct an image with varying degree of compression by using PCA. Can the reconstruction error ever be zero?
3. Assume you are given a new, slightly different, image of a person within the dataset. How could you recognize which person of the dataset it probably is? (Hint: Use the projections.)

References

- [1] J. Shlens. A tutorial on principal component analysis. *arXiv preprint arXiv:1404.1100*, 2014. URL <https://arxiv.org/abs/1404.1100>.
- [2] M. Turk and A. Pentland. Eigenfaces for recognition. *Journal of cognitive neuroscience*, 3(1):71–86, 1991.